

COAL

Lecture 5

Data Types

- **DB (Define Byte):** Allocates space for a byte (8 bits) or initializes it with a value.
 - Value DB 10 (value is name of variable initialize to 10)
- **DW (Define Word):** Allocates space for a word (usually 16 bits) or initializes it with a value.
 - Num DW 1234
- **DD (Define Doubleword):** Allocates space for a doubleword (usually 32 bits) or initializes it with a value.
 - largeNum DD 100000
- **DQ (Define Quadword):** Allocates space for a quadword (usually 64 bits) or initializes it with a value.
 - bigNum DQ 123456789012345

Variable Declaration and Initialization

- Defines a variable and reserves memory for it in the data segment.
- Syntax: `<variable_name> <directive> <value>`
- Assigns an initial value to the variable.
- The ? symbol is used when you want to **reserve memory** for a variable but **do not assign a value initially**. The reserved space will contain undefined (garbage) values until assigned later in the program.
 - `<variable_name> <directive> ?`

Examples

- message db 'Hello', 0
- number dw 5
- buffer db 10 ?
- count dd 1000
- data dw ?

db is used for 8-bit data (characters, small numbers).

dw is used for 16-bit data (larger numbers, addresses).

dd is used for 32-bit data (larger values or 32-bit operations).

Data Segment Handling

- `.data`
 - `; declare and initialize variables`
- `.code`
 - `MOV AX, @data ; Load data segment address into AX`
 - `MOV DS, AX ; Set DS to point to the data segment`
 - `; rest of the code`
- `.exit`

ALU Operations on Variables

- ADD AX, num1
- SUB CX, num2
- INC counter
- XOR AX, data
- AND AX, mask
- MUL num1
- NEG abc

Arrays

- An array is a collection of data elements of the same type, stored in consecutive memory locations.
- In 8086 assembly, arrays are typically defined in the **.data** segment.
 - **Base address:** The address of the first element of the array.
 - **Element size:** The size of each element (byte, word, etc.).
 - **Indexing:** Elements are accessed using an index (offset) relative to the base address.

Defining Arrays in 8086

- Use DB to define array
 - .data
 - array DB 10, 20, 30, 40, 50
- Use DW to define array
 - .data
 - array DW 100, 200, 300, 400

Accessing Array Elements

- To access array elements, you must calculate the memory address (offset) based on the array's base address and the index.
- Byte array (first element)
 - `MOV SI, 0 ; Set SI to index 0`
 - `MOV AL, [array + SI] ; AL = array[0]`
 - `INC SI ;move to second element`
- Word array (second element)
 - `MOV SI, 2 ; Set SI to index 1 (word = 2 bytes)`
 - `MOV AX, [array + SI] ; AX = array[1] (word)`
 - `ADD SI, 2 ;move to third element`

Cont.

- **Accessing Multiple Elements**
 - MOV SI, 0
 - MOV AL, [array + SI] ; AL = array[0]
 - INC SI
 - MOV AL, [array + SI] ; AL = array[1]

Using Loop to Process Arrays

```
.data
array DW 100, 200, 300, 400, 500 ; Array of 5 words (16-bit)
sum   DW 0                       ; Variable to store the sum of elements

.code
MOV SI, 0                        ; Set SI to 0 (start at the first element)
MOV CX, 5                        ; Set loop counter (CX = 5, for 5 elements)

process_array:
    MOV AX, [array + SI] ; Load the current element of the array into AX
    ADD [sum], AX         ; Add AX (the current element) to sum
    ADD SI, 2             ; Move to the next word (2 bytes ahead)
    LOOP process_array    ; Repeat the loop, decrementing CX each time

; After the loop ends, the sum will contain the sum of the array elements
```

Modifying Array Elements

```
MOV SI, 2           ; Set SI to index 2 (3rd element)
MOV [array + SI], 100 ; Set array[2] = 100
```

```
MOV CX, 5           ; Set loop counter for 5 elements
MOV SI, 0           ; Start at the first element
modify_loop:
    MOV AL, [array + SI] ; Load current element into AL
    ADD AL, 10           ; Add 10 to the element
    MOV [array + SI], AL ; Store the modified element back
    INC SI              ; Move to the next element
    LOOP modify_loop    ; Repeat for all elements
```

Jump Instruction

- **Jump** instructions alter the **sequence** of program execution by causing a branch to a different instruction in the program.
- **Unconditional Jump:** Always jumps to the target.
- **Conditional Jump:** Jumps based on meeting a certain condition (like comparison results; if/else).

Unconditional Jump

- These instructions always jump to a target address or label, regardless of any conditions.
- Syntax: JMP label
 - Will jump to the specified label regardless of any condition

Distance wise JMP

- JMP short
 - Jumps to a nearby address
 - Used for small distances in code.
- JMP near
 - Jumps within a range of 32K bytes (nearby memory addresses)
- JMP far
 - Used to jump to a different **segment**. It specifies both a segment and an offset.
 - JMP FAR [segment:offset]

CMP Instruction

- Compares two operands by subtracting the second operand from the first but does not store the result
- Only updates status flags after the execution
- Typically used to set the condition flags for conditional jumps
- CMP destination, source
 - CMP AX, BX ; Compare AX and BX (sets flags based on AX - BX)
 - AX will remain unaffected after execution

Test Instruction

- Performs a bitwise AND operation between two operands but does not store the result. It only sets or clears the status flags based on the result.
- Commonly used to check if specific bits in a register or memory location are set.
- TEST destination, source
 - TEST AL, 1 ; Test if the least significant bit of AL is set (sets flags based on AL AND 1)
 - AL will remain unaffected after execution

Conditional Jumps

- Conditional jump instructions evaluate a specific condition (based on flags set by previous instructions like CMP or TEST before jumping).
- Syntax: J<condition> label

JE / JZ

- Jump if equal
- Jump if zero
- ZF = 1 in both cases
- Example:
 - CMP AX, BX ; Compare AX with BX
 - JE equal ; Jump to “equal” if AX equals BX (ZF=1)
- JNE / JNZ

Jump – Single Flag

Instruction	Description	Condition	Opposite Instruction
JZ JE	Jump if Zero Jump if Equal	ZF = 1	JNZ JNE
JC JB JNAE	Jump if Carry Jump if Below Jump if Not Above Equal	CF = 1	JNC JNB JAE
JS	Jump if Sign.	SF = 1	JNS
JO	Jump if Overflow.	OF = 1	JNO
JPE/ JP	Jump if Parity Even.	PF = 1	JPO
JNZ JNE	Jump if Not Zero Jump if Not Equal	ZF = 0	JZ JE
JNC JNB JAE	Jump if Not Carry Jump if Not Below Jump if Above Equal	CF = 0	JC JB JNAE
JNS	Jump if Not Sign.	SF = 0	JS
JNO	Jump if Not Overflow.	OF = 0	JO
JPO JNP	Jump if Parity Odd Jump if No Parity	PF = 0	JPE JP

Jump – Unsigned Comparison

Instruction	Description	Condition	Opposite Instruction
JE JZ	Jump if Equal (=). Jump if Zero.	ZF = 1	JNE JNZ
JNE JNZ	Jump if Not Equal (<>). Jump if Not Zero.	ZF = 0	JE JZ
JA JNBE	Jump if Above (>). Jump if Not Below or Equal (not <=).	CF = 0 and ZF = 0	JNA JBE
JB JNAE JC	Jump if Below (<). Jump if Not Above or Equal (not >=). Jump if Carry.	CF = 1	JNB JAE JNC
JAE JNB JNC	Jump if Above or Equal (>=). Jump if Not Below (not <). Jump if Not Carry.	CF = 0	JNAE JB
JBE JNA	Jump if Below or Equal (<=). Jump if Not Above (not >).	CF = 1 or ZF = 1	JNBE JA

Jump – Signed Comparison

Instruction	Description	Condition	Opposite Instruction
JE JZ	Jump if Equal (=). Jump if Zero.	ZF = 1	JNE JNZ
JNE JNZ	Jump if Not Equal ($\neq$). Jump if Not Zero.	ZF = 0	JE JZ
JG JNLE	Jump if Greater ($>$). Jump if Not Less or Equal (not $\leq$).	ZF = 0 and SF = OF	JNG JLE
JL JNGE	Jump if Less ($<$). Jump if Not Greater or Equal (not $\geq$).	SF $\neq$ OF	JNL JGE
JGE JNL	Jump if Greater or Equal ($\geq$). Jump if Not Less (not $<$).	SF = OF	JNGE JL
JLE JNG	Jump if Less or Equal ($\leq$). Jump if Not Greater (not $>$).	ZF = 1 or SF $\neq$ OF	JNLE JG

Looping with Conditional Jumps

```
MOV CX, 5          ; Set loop counter to 5
loop_start:
    ; Some operations
    DEC CX          ; Decrease CX
    JNZ loop_start  ; Jump back to loop_start if CX != 0
```

Bit Manipulation

- Bit manipulation involves operations that directly modify individual bits of a binary number.
 - Flipping bits (0 to 1 and vice versa)
 - Combine bits in different ways
 - Shifting bits left or right
- Purpose: enhance efficiency and performance

Basic Bitwise Operators

- AND (&): Sets each bit to 1 if both corresponding bits are 1
- OR (|): Sets each bit to 1 if at least one of the corresponding bits is 1
- XOR (^): Sets each bit to 1 if only one of the corresponding bits is 1
- NOT (~): Inverts all bits (flips 0 to 1 and 1 to 0)

Shift Operators

- Left Shift Operation ($\ll$)
 - Moves all bits in a number to the left by a certain number of positions
 - Zeros are added to the right side as the bits are shifted left
 - The leftmost bits are discarded if they go beyond the fixed size of the bit representation
 - For example:
 - $5 \ll 1 = 1010$ (mul 2^1)
 - $3 \ll 2 = 1100$ (mul 2^2)
 - $65535 \ll 1 = 11111111111111110$ (leftmost 1 is discarded because of overflow)
 - Multiplication by power of 2

Logical Right Shift

- Done in unsigned numbers
- Bits are moved to the right
- Zeros are added on the left
- For example:
 - $\text{FF00} \gg 1: 1111111100000000 \gg 1 = 0111111110000000 = 7\text{F00}$
 - $18 \gg 3: 10010 \gg 3 = 00010 = 2$
 - $5 \gg 2: 101 \gg 2 = 001 = 1$
- Division by powers of 2

Arithmetic Right Shift

- For signed numbers only
- Leftmost 1 is preserved to maintain the sign in negative numbers
- 1s are added at left side instead of 0s
- For example:
 - $-5 \gg 2$: $1011 \gg 2 = 1110 = -2$ 11**10** - -
 - $-1000 \gg 3$: $1011100000 \gg 3 = 1111011100 = -125$ - - -
 - $0xA0 \gg 2$: $10100000 \gg 2 = 11101000 = 0xE8$ 11101000 - -

Rotations

- Rotate Left (ROL):
 - Shifts all bits of a value to the left
 - The leftmost bit (the most significant bit) is moved into the rightmost bit position (the least significant bit)
 - The other bits are shifted one position to the left
 - For Example:
 - 0001 1101 (29) = 0011 1010 (58) 0011 1010
 - 0101 1100 (92) = 1011 1000 (184)
 - 1100 1011 (203) = 1001 0111 (151)

Rotations

- Rotate Right (ROR):
 - Shifts all bits of a value to the right
 - The rightmost bit (the least significant bit) is moved into the leftmost bit position (the most significant bit)
 - The other bits are shifted one position to the right
 - For Example:
 - 0001 1101 (29) = 1000 1110 (142) 1000 1110 **1**000 1110
 - 0110 0101 (101) = 1011 0010 (178)
 - 1111 0001 (241) = 1111 1000 (232)